Automated Judging of LLM-based Smart Contract Security Auditors

Viraaji Mothukuri, Reza M. Parizi
Decentralized Science Lab, College of Computing and Software Engineering, Kennesaw State University, GA, USA
vmothuku@students.kennesaw.edu, rparizi1@kennesaw.edu

2025 IEEE International Conference on Blockchain and Cryptocurrency (ICBC) | 979-8-3315-4135-4/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICBC64466.2025.11114647

***Abstract*—The rise of Large Language Model-based smart contract auditors, supplementing manual audits, lacks standardized evaluation methods. This creates considerable uncertainty about their reliability and effectiveness in identifying vulnerabilities. This gap is particularly critical as smart contract exploits continue to result in billion-dollar losses and the complexity of blockchain systems grows exponentially. Our research introduces `smartJudge`, a comprehensive evaluation framework that systematically assesses the capabilities of LLM-based smart contract auditors through multi-dimensional analysis. To develop `smartJudge`, we first created an agenticRAG architecture that leverages agents to acquire in-depth knowledge and build expertise on smart contract auditing, vulnerabilities, and best practices. We then developed a distilled LLM judge model that efficiently processes auditor outputs through strategic evaluation points to evaluate how well auditors detect vulnerability patterns, understand security research, and identify complex vulnerability chains. Finally, we constructed a benchmark suite with evaluation metrics providing a standardized way to measure auditor performance across diverse vulnerability types. Testing `smartJudge` on the representative LLM-based auditors revealed critical gaps in vulnerability detection capabilities, particularly in complex attack vectors and novel exploit patterns. Our framework provides insights for improving automated auditing tools and establishes the first systematic methodology for evaluating LLM-based smart contract auditors, addressing a crucial bottleneck in blockchain security tools.**

***Index Terms*—Blockchain, Smart Contracts, Security Auditing, Large Language Models (LLMs), Graph-based Analysis, LLM Agents, Evaluation Framework, agenticRAG, Vulnerability Detection, LLM-as-a-judge.**

I. Introduction

The exponential growth of blockchain technology, now managing billions in digital assets [1]–[3], has positioned smart contracts as critical components of decentralized systems. While this rapid adoption demonstrates the technology's potential, it has also exposed significant vulnerabilities [4], resulting in catastrophic financial losses [5]. Traditional security approaches, including manual audits and automated analysis tools [6], are increasingly challenged in keeping pace with the complexity of smart contracts and the rapidly evolving threat landscape. Large Language Models (LLMs) have emerged as a promising solution for code comprehension [7]–[9] and vulnerability detection across various domains. Consequently, their use in smart contract security auditing has attracted growing interest from both researchers and industry, leading to the development of several LLM-based tools and approaches [10]–[15] capable of rapidly auditing smart contracts, unlike traditional manual reviews.

Evaluating LLM-based security tools presents significant challenges, including inconsistent assessment performance and vulnerability to prompt engineering manipulations [16]–[18]. Recent studies have documented error rates varying from 15-40% depending on the testing methodology [19], [20]. These issues are particularly critical in smart contract auditing, where false negatives can result in severe financial losses, and false positives can drain valuable auditing resources. To address these challenges, researchers have begun exploring alternative evaluation frameworks. The "LLM as a Judge" paradigm, where specialized LLMs evaluate the outputs of other security-focused models, has emerged as a promising approach [21]–[23]. With recent advances in LLMs, these systems are increasingly being adopted as automated evaluators or "judges" across various domains, offering structured frameworks for assessment based on their strong capabilities in natural language understanding and generation.

Our research addresses the critical challenges in evaluating LLM-based smart contract auditing by introducing a new instance of the LLM as a Judge paradigm, representing the first systematic approach to using language models as evaluation mechanisms in blockchain security audits. Building on this innovation, we propose a three-pronged approach that balances efficiency, accuracy, and reliability. First, we enhance domain expertise in our `smartJudge` through specialized knowledge distillation techniques [24], strategically combining multiple models distilled for different security evaluation tasks, enabling more informed assessments while also addressing computational overhead and inconsistency issues common in larger general-purpose models. Second, we address the context awareness limitation by introducing a novel agenticRAG [25], [26] that incorporates the GraphRAG framework [27] that captures the intricate relationships between smart contract components by incorporating structural and semantic contract information. The framework's graph-structured knowledge representation allows tracking complex interactions and dependencies that traditional approaches often miss. Finally, we establish a standardized evaluation methodology through our comprehensive benchmark suite. This suite provides comprehensive metrics for assessing both technical accuracy and impact considerations, enabling objective comparisons between different auditing approaches.

II. Related Work

Integrating LLMs into smart contract auditing has led to the development of various innovative tools and frameworks, each tackling unique aspects of vulnerability detection, compliance verification, and security analysis. For instance, AuditGPT [28] employs Task Specific Compliance for ERC standard verification, while SMARTINV [29] implements Contextual Enhancement through code documentation integration. GPTScan [12] utilizes Comparative Dataset Based evaluation with Web3Bugs datasets, comparing findings to human-audited results. GPTLens [30] adopts Role-Based Validation with auditor-critic dynamics to balance false positives and true positives. LLM4Vuln [31] performs Controlled Experimentation by isolating variables like prompt design across multiple scenarios. LLMSmartSec [11] introduces a multi-perspective analysis framework using annotated Control Flow Graphs as reusable knowledge bases. LLM SmartAudit [32] leverages multiple specialized LLM agents and employs Dual Dataset Validation to assess performance in controlled and real-world scenarios. Smartguard [33] combines traditional static detection with LLM-enhanced semantic analysis, while iAudit [34] employs Fine Tuning Precision Testing with a two-stage vulnerability detection framework. SmartLLMSentry [35] utilizes Curated Dataset Assessment with specialized vulnerability datasets for in-context learning. This inconsistency in evaluation approaches hampers direct comparisons between tools

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 20:39:10 UTC from IEEE Xplore. Restrictions apply.
979-8-3315-4135-4/25/$31.00 ©2025 IEEE

and raises concerns about their practical reliability in real-world applications, highlighting the need for standardized benchmarks.

III. Proposed Approach

Figure 1 illustrates the architecture of the proposed approach, `smartJudge`. The **Distillation Pipeline** forms the foundational learning component of the architecture. The *Teacher LLM Model* serves as the knowledge source, transferring specialized security expertise to smaller models. The *SLM* (Small Language Models) [36], [37] represent distilled versions that maintain essential capabilities while improving computational efficiency. Distillation Pipeline empowers the **smartJudge Agents**, which are: the *Pattern Detection Agent* for identifying vulnerability patterns, the *Technical Validation Agent* for deep code analysis, and the *Remediation Assessment Agent* for evaluating security fixes. At the center of the architecture lies the **Neo4J Knowledge Base**, serving as a comprehensive repository of security patterns, vulnerability patterns, audit rules, scientific research works, historical security data, and many more. This graph-based database connects the distillation, agenticRAG, and judging the judge pipelines, enabling knowledge representation and retrieval. The **agenticRAG Pipeline** facilitates the interaction between this knowledge base and various system components, enabling dynamic knowledge updates and efficient information sharing. The **Judging the Judge pipeline** is supported by three key components: the *Benchmark Dataset* providing reference data of evaluation metrics, *Quality Assurance* mechanisms ensuring evaluation accuracy, and *Judgment Validation* processes verifying assessment results. This ensures that we test the technical performance of evaluation systems and their ability to produce consistent, unbiased, and meaningful assessments across diverse contexts and use cases by using evaluation metrics that measure reliability, validity, accuracy, bias detection, and efficiency parameters to comprehensively validate the judge's capabilities.

The **Evaluation Benchmark Rubric** encompasses multiple critical metrics, including *Bias Detection*, *Vulnerability Detection Rate*, *Code Coverage*, and *Root Cause Analysis Depth*. Additional performance indicators include *Time to Audit/Detect*, *Processing Time per Contract*, *Cross-contract Vulnerability Detection*, *Contextual Understanding*, *Confidence Score Accuracy*, and *Bias assessment*. These metrics collectively ensure a comprehensive evaluation of smart contract auditing tools. Finally, the `smartJudge` is an LLM and the core component of our proposed approach that orchestrates all the pipelines. It represents the core evaluation component, processing results from *LLM Powered Auditors*. The architecture demonstrates a well-orchestrated flow from knowledge distillation to practical evaluation, incorporating multiple feedback loops and validation steps.

agenticRAG Pipeline: Our proposed approach introduces dynamic knowledge expansion through multi-agent coordination, enabling LLM-based judges to maintain a comprehensive and current understanding of emerging security patterns. Each agent maintains expertise in distinct domains, including security patterns, academic research, and smart contract vulnerabilities . This specialization enables broad coverage and deep domain expertise, addressing a critical limitation of traditional RAG implementations that often sacrifice depth for breadth. Central to our approach is the dynamic knowledge integration mechanism that operates continuously across multiple channels. The system integrates emerging academic research and analyzes new security patterns as they emerge. This multi-source approach ensures comprehensive coverage while implementing rigorous cross-validation to maintain knowledge accuracy. Our framework extends traditional knowledge representation by incorporating versioning of vulnerability patterns, enabling tracking of pattern evolution and relationship modeling through graph-based representations.

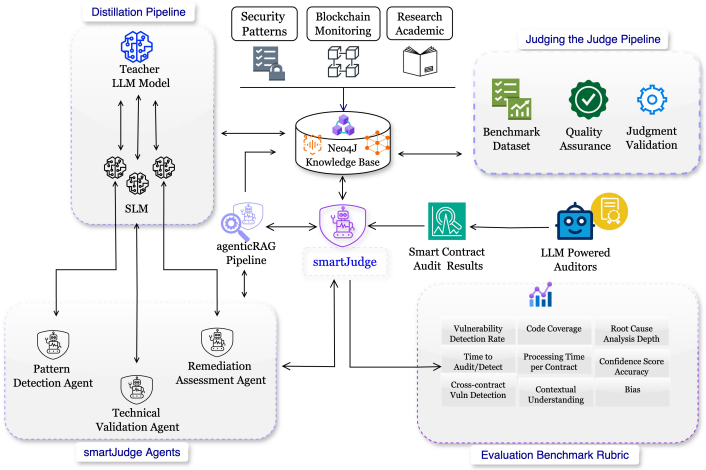


Fig. 1. The architecture of the proposed approach

Knowledge Distillation Process: It implements a teacher-student paradigm optimized for smart contract security analysis. The *Teacher LLM Model* acts as the knowledge source, transferring critical security expertise to smaller *SLM* models through a carefully orchestrated distillation pipeline. This process preserves essential vulnerability detection and analysis capabilities while significantly reducing computational overhead. The *Security Knowledge Transfer* phase employs temperature-scaled soft targets to distill specialized knowledge from the teacher model to student models. The teacher model, trained extensively on smart contract vulnerability datasets, including recent exploits and established patterns, transfers this expertise through targeted optimization techniques. This approach enables the student models to maintain high accuracy in security pattern recognition despite their reduced size. Through *Specialized Agent Training*, each agent undergoes focused distillation tailored to its specific role. The *Pattern Detection Agent* specializes in rapid vulnerability pattern matching, while the *Technical Validation Agent* focuses on comprehensive code analysis and verification. The *Remediation Assessment Agent* specializes in evaluating and validating proposed security fixes.

The framework implements progressive knowledge distillation, where student models are iteratively trained on increasingly complex security patterns. The approach utilizes selective attention mechanisms and layer-wise knowledge transfer. The distillation incorporates dynamic performance optimization techniques, including adaptive batch sizing, selective layer pruning, and distributed knowledge caching mechanisms. Integration with the *agenticRAG Pipeline* ensures these distilled models remain current with emerging vulnerability patterns and attack vectors through knowledge updates. This comprehensive strategy achieves a substantial reduction in computational requirements while maintaining high accuracy, enabling rapid deployment and scaling of audit evaluation capabilities while ensuring robust security assessment quality.

SLM-based Audit Evaluation Agents: Our approach introduces specialized multi-agents designed to support `smartJudge`, a Large Language Model (LLM) acting as the primary evaluator of smart contract audit results. This enhances `smartJudge`'s decision-making capabilities through three specialized agents. The *Pattern Detection Agent* serves as `smartJudge`'s pattern recognition specialist, analyzing audit outputs for consistency in vulnerability identification across different auditors. It maintains a comprehensive vulnerability

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 20:39:10 UTC from IEEE Xplore. Restrictions apply.

pattern database, enabling it to assess the completeness and accuracy of reported findings. The *Technical Validation Agent* augments `smartJudge`'s technical reasoning by performing in-depth code analysis validation. When auditors present conflicting technical assessments, this agent provides them with detailed control flow and state transition analyses to support informed decision-making. The *Remediation Assessment Agent* assists `smartJudge` in evaluating the quality and feasibility of proposed fixes. It validates remediation strategies suggested by different auditors and implementation complexity to help rank solution effectiveness. These agents operate as `smartJudge`'s specialized assistants, each contributing domain-specific insights to a shared knowledge base. `smartJudge` leverages this collective intelligence to make final determinations on audit quality, arbitrate between conflicting findings, and assess the effectiveness of different smart contract auditors.

agenticRAG Integration: Finally, the `smartJudge` agents, integrate seamlessly with the broader agenticRAG pipeline. As the RAG layer updates its knowledge graph with new exploit data or academic findings, the SLM agents gain access to this updated state. This cohesive integration ensures that the entire pipeline, from retrieval to analysis, validation, and remediation recommendation, remains state-of-the-art and responsive to the evolving landscape of smart contract security.

Benchmark Evaluation Metrics: Our evaluation framework establishes a comprehensive methodology for assessing LLM-powered smart contract auditors across critical dimensions: vulnerability analysis, reasoning quality, contextual understanding, comparative performance, and novel detection capabilities. The metrics are designed to capture quantitative performance through precise mathematical formulations (detection precision, recall) and qualitative aspects through expert evaluation scales. Notably, this framework emphasizes the importance of measuring not only traditional security assessment metrics but also the LLM's ability to provide coherent explanations, understand complex DeFi (Decentralized Finance) [38], [39] mechanisms, and identify novel vulnerability patterns. The comparative performance metrics, particularly Cohen's Kappa [40] and Scott's Pi [41], offer correlation with expert audits and provide crucial benchmarking against human performance, while the novel detection dimension addresses the system's capability to advance the state of smart contract security through the identification of previously unknown vulnerability patterns.

We have developed a comprehensive rubric (presented in the project's Github [42]) that provides a qualitative framework for evaluating the performance and capabilities of LLM-powered smart contract auditors across the above dimensions. Each category, ranging from Vulnerability Detection and Advanced Efficiency to Novel Pattern Detection and Implementation Framework, features metrics paired with four graded performance tiers: Exemplary, Proficient, Developing, and Needs Improvement. By translating numerical metrics and procedural benchmarks into descriptive criteria, the rubric enables a holistic understanding of system strengths and weaknesses, fostering a clear path for improvement. It helps to swiftly gauge where an auditing tool excels, where it meets acceptable standards, and where it may require strategic enhancements. Ultimately, this rubric is a guiding instrument to ensure consistent, meaningful, and actionable evaluations of LLM-driven security auditing solutions.

Judging the Judge: The effectiveness of the `smartJudge` in assessing LLM-powered smart contract auditors will be determined through: consistent rubric application across models, comparative analysis of candidate judges, human feedback integration, adoption of emerging metrics, and periodic benchmark recalibration. This enhances transparency, accountability, and continuous improvement of the judges in assessing and ensures that we select the most effective LLM as `smartJudge`.

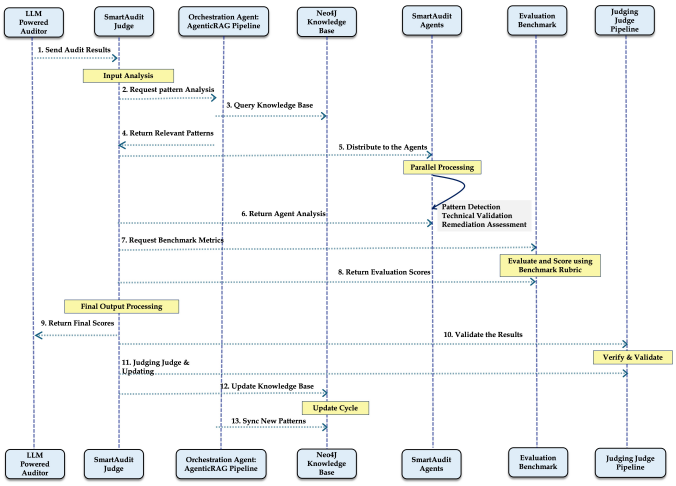


Fig. 2. Workflow of `smartJudge`

Figure 2 illustrates the proposed approach workflow. The workflow begins when the results of LLM Powered Auditor are submitted to the `smartJudge`, which initiates an input analysis phase. The judge then interfaces with the agenticRAG Pipeline, requesting pattern analysis, which in turn queries the Neo4J Knowledge Base to retrieve relevant security patterns and historical audit data. This knowledge retrieval stage ensures the evaluation is grounded in established security patterns and best practices. Once patterns are returned, the system distributes the analysis task to `smartJudge` Agents, which perform parallel processing across three specialized domains: pattern detection, technical validation, and remediation assessment. After agent analysis is complete, results are measured against the Evaluation Benchmark component using standardized metrics for vulnerability detection accuracy, code coverage, and efficiency using a structured benchmark rubric. The `smartJudge` processes final outputs before returning them to the auditor, followed by meta-evaluation through the Judging Judge Pipeline and knowledge base updates to enhance future evaluations.

IV. Evaluation and Results

Our experimental workspace leverages an NVIDIA RTX 2080 Super GPU with 64.0 GB RAM and a 256-bit memory interface. The development environment integrates graph database Neo4J v1.5.9 [43] for vulnerability pattern storage, Python 3.8+ with CUDA 11.4 acceleration. We utilize distilled fine-tuned SLMs, and for LLMs, the GPT-3.5 [44] and GPT-4 [45] using OpenAI APIs, and the Claude Sonnet3.5 model from Anthropic [46]. The complete framework for evaluating smart contract audit tools, including evaluation metrics and comparison utilities, is available in our `GitHub` repository [47].

Our framework employs a dual evaluation approach tailored to tool accessibility. For tools without GitHub sources, where we can't perform direct code analysis, we provide standardized evaluation by mapping documented capabilities to our benchmark suite and validating published results against known vulnerability datasets. The `smartJudge` framework parses these materials using agenticRAG, extracting architectural details, methodologies, and performance metrics. This enables rubric-based assessment of vulnerability detection methods, efficiency benchmarks, and novel feature identification. This ensures fair assessment even without source code access. For tools with access to code repositories, our framework performs comprehensive analysis through several key steps: direct integration with their codebase using custom adapters executing the tools; measuring key performance

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 20:39:10 UTC from IEEE Xplore. Restrictions apply.

metrics; generating detailed performance reports with objective scoring across all metrics; and enabling side-by-side comparisons between different auditing approaches. This methodology ensures consistent benchmarking standards across the entire ecosystem while accommodating commercial closed-source tools and open-source projects.

We leverage evaluation NLP metrics, including the BLEU [48] and the ROUGE [49] scores, to evaluate the quality of vulnerability explanations to provide baseline measurements, while precision, recall, and F1-scores quantify detection accuracy. For measuring agreement with human auditors [50], we employ Cohen's Kappa coefficients for contract-specific metrics, including vulnerability pattern detection rates and cross-contract vulnerability detection accuracy. These metrics provide quantitative validation for our qualitative rubric assessments presented in Table I. The overall results demonstrate the effectiveness of our evaluation framework across both linguistic and technical dimensions of smart contract security analysis.

The visualization of seven auditors [11], [12], [28]–[31], [33] performance in Figure 3 clearly illustrates that while most LLM-powered auditors are performing at a "Proficient" or high "Developing" level in the area of Vulnerability Detection, there remains a noticeable gap in Advanced Efficiency and Novel Pattern Detection metrics across the board. Many tools show strengths in identifying known vulnerabilities but have not matured enough to demonstrate robust zero-day discovery capabilities or optimized resource usage. The auditors that compare results against human auditors or utilize specialized datasets appear more aligned with practical auditing scenarios, suggesting that real-world benchmarking and continuous human-validation loops may be integral to achieving "Exemplary" performance. Overall, the results highlight both the progress made in precision and recall metrics and the need for further development in advanced efficiency, coverage comprehensiveness, and innovative threat prediction to fully realize the potential of LLM-powered smart contract auditors.

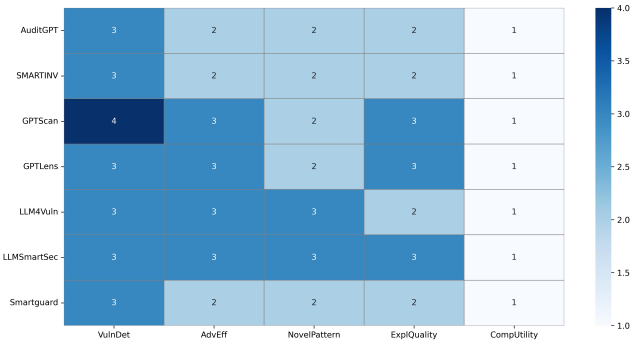


Fig. 3. Rubric-Based Evaluation of LLM-based Smart Contract Auditors

The rubric Table I provides a comprehensive checklist view across 31 key metrics organized into 8 critical categories, from vulnerability detection to operational efficiency. This structured approach effectively maps out both GPTScan [12] and LLMSmartSec's [11] capabilities, using a clear performance scale from Needs Improvement ($< 70\%$) to Exemplary ($> 95\%$). The detailed breakdown helps identify each tool's strengths, like GPTScan's proficiency in zero-day discovery and LLMSmartSec's exemplary reproducible environments. This systematic evaluation framework ensures no critical aspect of smart contract analysis tools goes unexamined.

As illustrated in Figure 4, after integrating agenticRAG, the `smartJudge` demonstrated a marked improvement in domain knowledge completeness, covering a wider range of current and emerging vulnerabilities. Contextual understanding of complex

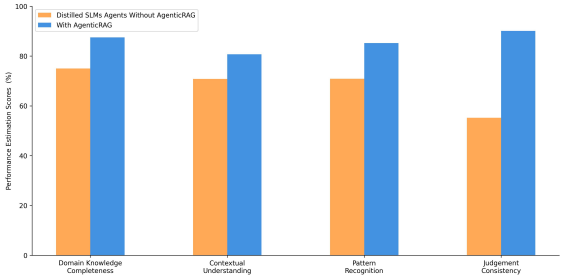


Fig. 4. Impact of Agentic RAG on `smartJudge`

exploit patterns improves as the judge can retrieve specialized documentation or recent security advisories, leading to more nuanced evaluations. While our distilled model already provided strong baseline performance, adding agenticRAG showed notable improvements. These improvements highlight agenticRAG's value in enhancing our framework's ability to judge LLM-based smart contract auditors more comprehensively and accurately.

TABLE I
BENCHMARK RUBRIC OF LLM-BASED SMART CONTRACT AUDITORS

Category	Metric	llmsmartsec [11]	gptscan [12]
Vulnerability Detection	Precision	✔✔✔	✔✔✔
	Recall	✔✔✔	✔✔
	False Positive Rate	✔✔✔	✔✔✔
	Detection Coverage	✔✔✔	✔✔
Advanced Efficiency	Resource Utilization	✔✔✔	✔✔✔
	Multi-Contract Scaling	✔✔	✔✔
	Memory Optimization	✔✔✔	✔✔✔
	API Call Efficiency	✔✔✔	✔✔✔
	Batch Processing	✔✔✔	✔✔✔
Novel Pattern Detection	Zero Day Discovery	✔✔	✔✔
	Pattern Evolution	✔✔	✔✔✔
	Exploit Chain ID	✔✔✔	✔✔
	Attack Vector Prediction	✔✔	✔✔
	Threat Recognition	✔✔	✔✔
Implementation Framework	Test Harnesses	✔✔✔	✔✔✔
	Reproducible Environments	✔✔✔✔	✔✔✔
	Version Control	✔✔✔	✔✔✔
	Metric Collection	✔✔✔	✔✔✔
	Result Validation	✔✔✔	✔✔✔
Explanation Quality	Clarity	✔✔✔	✔✔✔
	Root Cause Analysis	✔✔✔	✔✔✔
	Remediation Advice	✔✔✔	✔✔
Coverage Analysis	Code Coverage	✔✔✔	✔✔✔
	Cross-Contract Analysis	✔✔✔	✔✔✔
	Blind Spot Detection	✔✔✔	✔✔
Comparative Utility	Human Agreement	✔✔	✔✔
	Run Consistency	✔✔	✔✔
	Pattern Generalization	✔✔	✔✔
Operational Efficiency	Scalability	✔✔✔	✔✔✔
	Error Handling	✔✔✔	✔✔✔
	System Integration	✔✔✔	✔✔✔

Performance Levels:

- ✔✔✔✔- Exemplary: $> 95\%$ - Outstanding performance exceeding expectations
- ✔✔✔- Proficient: $85\% - 95\%$ - Strong performance meeting all requirements
- ✔✔Developing: $70\% - 84\%$ - Acceptable performance with room for improvement
- ✔Needs Improvement: $< 70\%$ - Performance below acceptable threshold

V. CONCLUSION AND FUTURE WORK

We proposed an LLM-as-a-Judge-based approach for evaluating LLM-powered smart contract auditors that addresses key challenges in automated security assessment. Our evaluation demonstrated that combining distilled models with agenticRAG significantly improves audit evaluation efficiency and accuracy. The innovative benchmark framework demonstrates the compelling potential for adaptation in assessing LLM-based systems in other security-critical domains, such as cybersecurity and compliance monitoring, contributing to more reliable automated security evaluation practices. For our future work, we plan to enhance the "Judging the Judge" pipeline by leveraging concurrency in `smartJudge`, optimizing cost and computational performance when stages run in parallel. A key focus will be testing how various LLMs perform as judges to evaluate potential improvements in overall accuracy and robustness. Furthermore, our roadmap includes developing a decentralized platform for collaborative benchmark creation and rolling out metrics to assess the financial impact of vulnerabilities in DeFi sector.

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 20:39:10 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] V. T. Truong, L. Le, and D. Niyato, "Blockchain meets metaverse and digital asset management: A comprehensive survey," *IEEE Access*, vol. 11, pp. 26 258–26 288, 2023.

[2] D. Bennet, L. Maria, Y. P. A. Sanjaya, and A. R. A. Zahra, "Blockchain technology: Revolutionizing transactions in the digital age," *ADI Journal on Recent Innovation*, vol. 5, no. 2, pp. 192–199, 2024.

[3] D. Dhillon, Diksha, and D. Mehrotra, "Smart contract vulnerabilities: Exploring the technical and economic aspects," in *Blockchain Transformations: Navigating the Decentralized Protocols Era*. Springer, pp. 81–91.

[4] S. M. Nzuva, "Revisiting blockchain technologies and smart contracts security: A pragmatic exploration of vulnerabilities, threats, and challenges," *Asian Journal of Research in Computer Science*, vol. 17, no. 7, pp. 11–30, 2024.

[5] A. Augusto, R. Belchior, M. Correia, A. Vasconcelos, L. Zhang, and T. Hardjono, "Sok: Security and privacy of blockchain interoperability," in *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, pp. 3840–3865.

[6] T. Durieux, J. F. Ferreira, R. Abreu, and P. Cruz, "Empirical review of automated analysis tools on 47,587 ethereum smart contracts," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, ser. ICSE '20. ACM, Jun. 2020.

[7] J. He and M. Vechev, "Large language models for code: Security hardening and adversarial testing," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '23. ACM, Nov. 2023. [Online]. Available: http://dx.doi.org/10.1145/3576915.3623175

[8] T. Ahmed and P. Devanbu, "Few-shot training llms for project-specific code-summarization," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '22. New York, NY, USA: Association for Computing Machinery, 2023.

[9] E. Nijkamp, H. Hayashi, C. Xiong, S. Savarese, and Y. Zhou, "Codegen2: Lessons for training llms on programming and natural languages," 2023.

[10] B. Boi, C. Esposito, and S. Lee, "Smart contract vulnerability detection: The role of large language model (llm)," *ACM SIGAPP Applied Computing Review*, vol. 24, no. 2, pp. 19–29.

[11] V. Mothukuri, R. M. Parizi, and J. L. Massa, "Llmsmartsec: Smart contract security auditing with llm and annotated control flow graph," in *2024 IEEE International Conference on Blockchain (Blockchain)*. IEEE, pp. 434–441.

[12] Y. Sun, D. Wu, Y. Xue, H. Liu, H. Wang, Z. Xu, X. Xie, and Y. Liu, "Gptscan: Detecting logic vulnerabilities in smart contracts by combining gpt with program analysis," 2023.

[13] I. David, L. Zhou, K. Qin, D. Song, L. Cavallaro, and A. Gervais, "Do you still need a manual smart contract audit?" *arXiv preprint arXiv:2306.12338*, 2023.

[14] Z. He, Z. Li, S. Yang, A. Qiao, X. Zhang, X. Luo, and T. Chen, "Large language models for blockchain security: A systematic literature review," *arXiv preprint arXiv:2403.14280*, 2024.

[15] C. Chen, J. Su, J. Chen, Y. Wang, T. Bi, Y. Wang, X. Lin, T. Chen, and Z. Zheng, "When chatgpt meets smart contract vulnerability detection: How far are we?" 2023.

[16] S. Shankar, J. D. Zamfirescu-Pereira, B. Hartmann, A. G. Parameswaran, and I. Arawjo, "Who validates the validators? aligning llm-assisted evaluation of llm outputs with human preferences," 2024. [Online]. Available: https://arxiv.org/abs/2404.12272

[17] B. Guo, X. Zhang, Z. Wang, M. Jiang, J. Nie, Y. Ding, J. Yue, and Y. Wu, "How close is chatgpt to human experts? comparison corpus, evaluation, and detection," 2023. [Online]. Available: https://arxiv.org/abs/2301.07597

[18] K. Zhou, Y. Zhu, Z. Chen, W. Chen, W. X. Zhao, X. Chen, Y. Lin, J.-R. Wen, and J. Han, "Don't make your llm an evaluation benchmark cheater," *arXiv preprint arXiv:2311.01964*, 2023.

[19] M. Mizrahi, G. Kaplan, D. Malkin, R. Dror, D. Shahaf, and G. Stanovsky, "State of what art? a call for multi-prompt llm evaluation," 2024. [Online]. Available: https://arxiv.org/abs/2401.00595

[20] C. Siska, K. Marazopoulou, M. Ailem, and J. Bono, "Examining the robustness of LLM evaluation to the distributional assumptions of benchmarks," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, L.-W. Ku, A. Martins, and V. Srikumar, Eds. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 10 406–10 421. [Online]. Available: https://aclanthology.org/2024.acl-long.560

[21] J. Gu, X. Jiang, Z. Shi, H. Tan, X. Zhai, C. Xu, W. Li, Y. Shen, S. Ma, H. Liu, Y. Wang, and J. Guo, "A survey on llm-as-a-judge." [Online]. Available: https://arxiv.org/abs/2411.15594

[22] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing *et al.*, "Judging llm-as-a-judge with mt-bench and chatbot arena," *Advances in Neural Information Processing Systems*, vol. 36, pp. 46 595–46 623, 2023.

[23] Z. Sollenberger, J. Patel, C. Munley, A. Jarmusch, and S. Chandrasekaran, "Llm4vv: Exploring llm-as-a-judge for validation and verification testsuites," in *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2024, pp. 1885–1893.

[24] X. Dong, Y. Fu, S. Diao, W. Byeon, Z. Chen, A. S. Mahabaleshwarkar, S.-Y. Liu, M. V. Keirsbilck, M.-H. Chen, Y. Suhara, Y. Lin, J. Kautz, and P. Molchanov, "Hymba: A hybrid-head architecture for small language models." [Online]. Available: https://arxiv.org/abs/2411.13676

[25] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," 2021. [Online]. Available: https://arxiv.org/abs/2005.11401

[26] C. Ravuru, S. S. Sakhinana, and V. Runkana, "Agentic retrieval-augmented generation for time series analysis," 2024. [Online]. Available: https://arxiv.org/abs/2408.14484

[27] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, "From local to global: A graph rag approach to query-focused summarization." [Online]. Available: https://arxiv.org/abs/2404.16130

[28] S. Xia, S. Shao, M. He, T. Yu, L. Song, and Y. Zhang, "Auditgpt: Auditing smart contracts with chatgpt."

[29] S. Wang, K. Pei, and J. Yang, "Smartinv: Multimodal learning for smart contract invariant inference," in *2024 IEEE Symposium on Security and Privacy (SP)*. Los Alamitos, CA, USA: IEEE Computer Society, may 2024, pp. 129–129.

[30] S. Hu, T. Huang, F. İlhan, S. F. Tekin, and L. Liu, "Large language model-powered smart contract vulnerability detection: New perspectives," 2023.

[31] Y. Sun, D. Wu, Y. Xue, H. Liu, W. Ma, L. Zhang, M. Shi, and Y. Liu, "Llm4vuln: A unified evaluation framework for decoupling and enhancing llms' vulnerability reasoning."

[32] Z. Wei, J. Sun, Z. Zhang, X. Zhang, M. Li, and Z. Hou, "Llm-smartaudit: Advanced smart contract vulnerability detection." [Online]. Available: https://arxiv.org/abs/2410.09381

[33] H. Ding, Y. Liu, X. Piao, H. Song, and Z. Ji, "Smartguard: An llm-enhanced framework for smart contract vulnerability detection," 01 2024.

[34] W. Ma, D. Wu, Y. Sun, T. Wang, S. Liu, J. Zhang, Y. Xue, and Y. Liu, "Combining fine-tuning and llm-based agents for intuitive smart contract auditing with justifications."

[35] O. Zaazaa and H. El Bakkali, "Smartllmsentry: A comprehensive llm based smart contract vulnerability detection framework," *Journal of Metaverse*, Oct. [Online]. Available: http://dx.doi.org/10.57019/jmv.1489060

[36] M. Javaheripi, S. Bubeck, M. Abdin, J. Aneja, S. Bubeck, C. C. T. Mendes, W. Chen, A. Del Giorno, R. Eldan, S. Gopi *et al.*, "Phi-2: The surprising power of small language models," *Microsoft Research Blog*, 2023.

[37] T. Schick and H. Schütze, "It's not just size that matters: Small language models are also few-shot learners," 2021. [Online]. Available: https://arxiv.org/abs/2009.07118

[38] C. R. Harvey, A. Ramachandran, and J. Santoro, *DeFi and the Future of Finance*. John Wiley & Sons, 2021.

[39] D. A. Zetzsche, D. W. Arner, and R. P. Buckley, "Decentralized finance (defi)," *Journal of Financial Regulation*, vol. 6, pp. 172–203, 2020.

[40] A. Doewes, N. A. Kurdhi, and A. Saxena, "Evaluating quadratic weighted kappa as the standard performance metric for automated essay scoring," in *Proceedings of the 16th International Conference on Educational Data Mining*. Bengaluru, India: International Educational Data Mining Society, July 2023, pp. 103–113.

[41] W. A. Scott, "Reliability of content analysis: The case of nominal scale coding," *The Public Opinion Quarterly*, vol. 19, no. 3, pp. 321–325, 1955. [Online]. Available: http://www.jstor.org/stable/2746450

[42] V. Mothukuri, "smartjudgerepo." [Online]. Available: https://github.com/dslmllab/Automated-Judging-of-LLM-based-Smart-Contract-Security-Auditors/blob/main/smartJudgeBenchmark/README.md

[43] Neo4J, "About neo4j desktop." [Online]. Available: https://neo4j.com/docs/desktop-manual/current/about-desktop/

[44] T. B. B. et.all, "Language models are few-shot learners," 2020.

[45] O. et.al, "Gpt-4 technical report."

[46] Anthropic, "Claude.ai." [Online]. Available: https://www.anthropic.com/research

[47] V. Mothukuri, "smartjudgerepo," Github, Accessed 2025. [Online]. Available: https://github.com/dslmllab/Automated-Judging-of-LLM-based-Smart-Contract-Security-Auditors

[48] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "Bleu: a method for automatic evaluation of machine translation," in *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics*, ser. ACL '02. USA: Association for Computational Linguistics, 2002, p. 311–318. [Online]. Available: https://doi.org/10.3115/1073083.1073135

[49] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*. Barcelona, Spain: Association for Computational Linguistics, Jul. 2004, pp. 74–81. [Online]. Available: https://aclanthology.org/W04-1013

[50] S. Banerjee and A. Lavie, "METEOR: An automatic metric for MT evaluation with improved correlation with human judgments," in *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*, J. Goldstein, A. Lavie, C.-Y. Lin, and C. Voss, Eds. Ann Arbor, Michigan: Association for Computational Linguistics, Jun. 2005, pp. 65–72. [Online]. Available: https://aclanthology.org/W05-0909

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 20:39:10 UTC from IEEE Xplore. Restrictions apply.